Millikan Oil Drop Lab Report

Jadon Tsai and Jeffery Tian

November 11th, 2025

1 Introduction

The purpose of this set of experiments is to determine the elementary charge of the electron and to confirm the quantization of electric charge. This is shown by comparing the forces on a moving charged oil droplet. The oil droplet experiences 3 forces (buoyancy is neglected in this experiment): a downwards gravitational force, an upwards electrical force, and an viscous friction force opposing the direction of motion. We used a Leybold-Heraeus apparatus, which adds charge to the oil droplets by friction, and then controls the velocity of the oil droplet via two charged plates powered by a variable power supply. We measure the velocity of the droplet and assume a spherical droplet to leave two unknowns; the radius and charge of the droplet. By measuring the same droplet at two different voltages (and therefore velocities), we can solve for both unknowns. We then took 52 trials and then found the greatest common divisor of the charges, which was determined to be $1.7 \times 10^{-19}C$. This differed from the accepted value of $1.6 \times 10^{-19}C$ by 6.12%

2 Equipment and Methodology

2.1 Equipment

The experimental setup consisted of a Leybold-Heraeus apparatus connected to a computer running LabVIEW software via a digital camera [1]. Oil droplets were squeezed in by a rubber bulb and their velocity was controlled by the potential difference between two plates, which was controlled by a variable power supply.

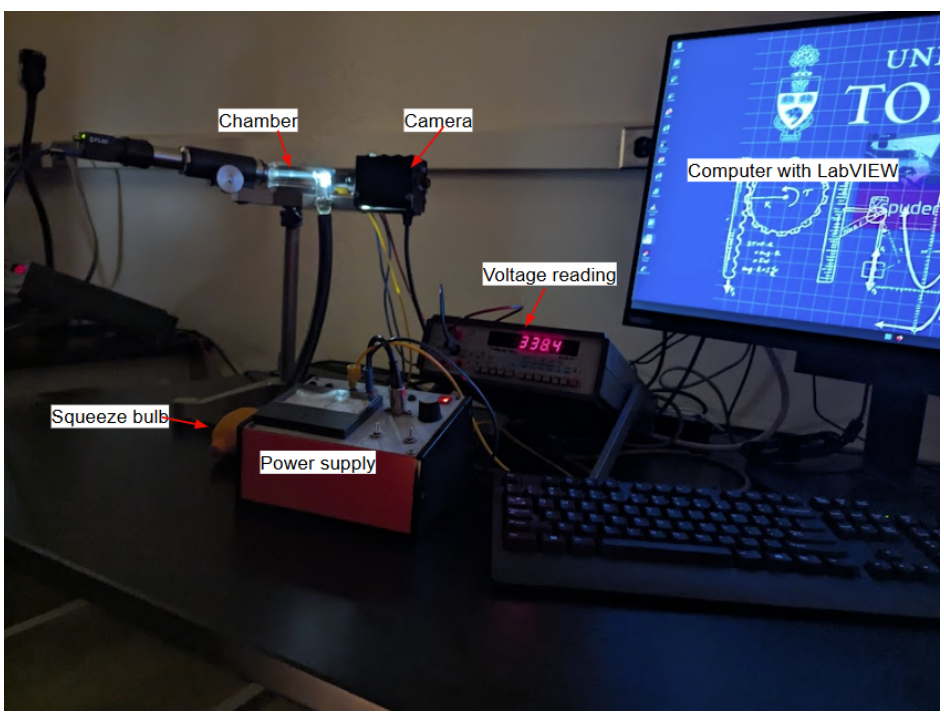


Figure 1: The labeled lab setup. The actual experiment was performed in the dark, the lights were turned on for clarity and illustrative purposes.

2.2 Methods

2.2.1 Lab Methods

The lab setup is shown in figure 1. Our data gathered in the lab was primarily gathered using a modified version method 2 from the lab manual [1] as we were unable to track the droplets with the either. We performed all experiments in a dark room. For all trials, the following steps were performed. We first squeezed the rubber bulb to send in the oil droplets. We waited for approximately 1 minute for the droplets of larger size to fall through, and then adjusted the voltage as required until we spotted a bright enough oil drop that was rising. We then pressed "Track image" and placed the tracking box around the oil drop of choice. We noted the rising voltage, and then decreased the voltage to where the droplet continued to drop at a low enough velocity that it could still be tracked. Data was then saved to an Excel spreadsheet.

2.2.2 Data processing methods

Data was taken from our own experiments as well as from one other group (Dazhou and Juncheng's group). In the github (also available in the appendix), we have noted their data in the files starting "group_dazhou" and our data in the files starting "group_jeffery". Data was taken from these files and then separated into occurrences of either rising droplets or falling droplets. As the tracking software sometimes moved in the opposite direction of motion (or occasionally stopped tracking) for a couple frames, we decided to use a monotonic filtering algorithm to remove position reversals [2] on the data. This ensured that all recorded droplet motion was unidirectional, which was what was physically observed in the lab for all experimental trials. This algorithm worked by removing values that were clearly erroneous due to software tracking issues (i.e. position values that either did not exist or implied sudden change in direction). This helped simplify the resulting velocity calculations. Discussion of the effect of this algorithm on results will be discussed in the results section. We then found velocity to be the slope of the distance time graph for each trial.

2.2.3 Calculation methods

To calculate the excess charge of the oil drops, there are two different available methods, each with their own respective equations. The first method uses the stopping voltage (V_{stop}) and the falling terminal velocity (v_d) of an electron, while the second uses the rising terminal velocity (v_u) with its associated voltage (V_u), as well as the falling terminal velocity (v_d). However, for a significant portion of our data, the falling terminal velocity was determined with a voltage (V_d) present, so some of our equations were modified to accommodate this difference.

2.2.4 Formulae for Method 1

For the first method, we start with the stopping voltage equation:

$$q\frac{V_{stop}}{d} = g(m_{oil} - m_{air})$$

Additionally, we use the equation for terminal velocity of a falling oil droplet, with the addition of an electric force generated by (V_d ¡ V_{stop}), and taking into account the modification of Stokes' Law presented by Cunningham:

$$q\frac{V_d}{d} + 6\pi r\nu v_d(1 + \frac{1.63l}{r})^{-1} = g(m_{oil} - m_{air}) \tag{1}$$

To derive a usable equation for q, we must first find a suitable equation for r in terms of the other measurable quantities and constants. To do so, we use the stopping voltage equation to remove the q dependence:

$$(\frac{g(m_{oil} + m_{air})d}{V_{stop}})(\frac{V_d}{d}) + 6\pi r\nu v_d(1 + \frac{1.63l}{r})-1 = g(m_{oil} - m_{air})$$

$$6\pi\nu v_d(r + 1.63l)-1 = g(m_{oil} - m_{air})(1 - \frac{V_d}{V_{stop}})$$

Using the assumption that the droplet is spherical, we can write both the masses as $\frac{4}{3}\pi r^3(\rho_{oil} - \rho_{air})$, we rewrite the equation and solve the resulting quadratic for r:

$$6\pi\nu v_d r(1 + 1.63l/r)^{-1} = \frac{4}{3}\pi r^3 g(\rho_{oil} - \rho_{air})(1 - \frac{V_d}{V_{stop}})$$

$$\frac{4}{3}\pi r^2(1 + 1.63l/r)g(\rho_{oil} - \rho_{air})(1 - \frac{V_d}{V_{stop}}) = 6\pi\nu v_d$$

$$r^2 + 1.63lr = \frac{9V_{stop}\ \nu v_d}{2g(V_{stop} - V_d)(\rho_{oil} - \rho_{air})}$$

$$r = \boxed{\sqrt{\frac{9V_{stop}\nu v_d}{2g(V_{stop} - V_d)(\rho_{oil} - \rho_{air})} + (0.815l)^2} - 0.815l}$$

Considering the simplified version of Stokes' Law (l = 0), and $V_d = 0$ (oil particle free fall), the quadratic reduces to:

$$r = \sqrt{\frac{9\nu v_d}{2g(\rho_{oil} - \rho_{air})}}$$

After obtaining a value of r from solving the quadratic, we plug it back into the stopping voltage equation and solve for q:

$$q = \frac{dg(m_{oil} - m_{air})}{V_{stop}}$$

$$q = \boxed{\frac{4\pi dg(\rho_{oil} - \rho_{air})r^3}{3V_{stop}}}$$

Plugging in the simplified value of r into this equation yields the result:

$$q = \frac{4\pi d(\rho_{oil} - \rho_{air})(\sqrt{\frac{18\nu v_d}{4g(\rho_{oil} - \rho_{air})}})^3}{3V_{stop}}$$

$$q = \frac{9\sqrt{2}\pi d\nu^{\frac{3}{2}} v_d^{\frac{3}{2}}}{g^{\frac{3}{2}}\sqrt{\rho_{oil} - \rho_{air}}V_{stop}}$$

Which takes on the expected form of $q = C_1\frac{v_d^{\frac{3}{2}}}{V_{stop}}$, with $C_1 = \frac{9\sqrt{2}\pi d\nu^{\frac{3}{2}}}{g^{\frac{3}{2}}\sqrt{\rho_{oil} - \rho_{air}}}$. As the formulae we determined for r and q agree with the expected formula for the simpler case, we will use both for the remainder of our lab when solving for q using this method.

2.2.5 Formulae for Method 2

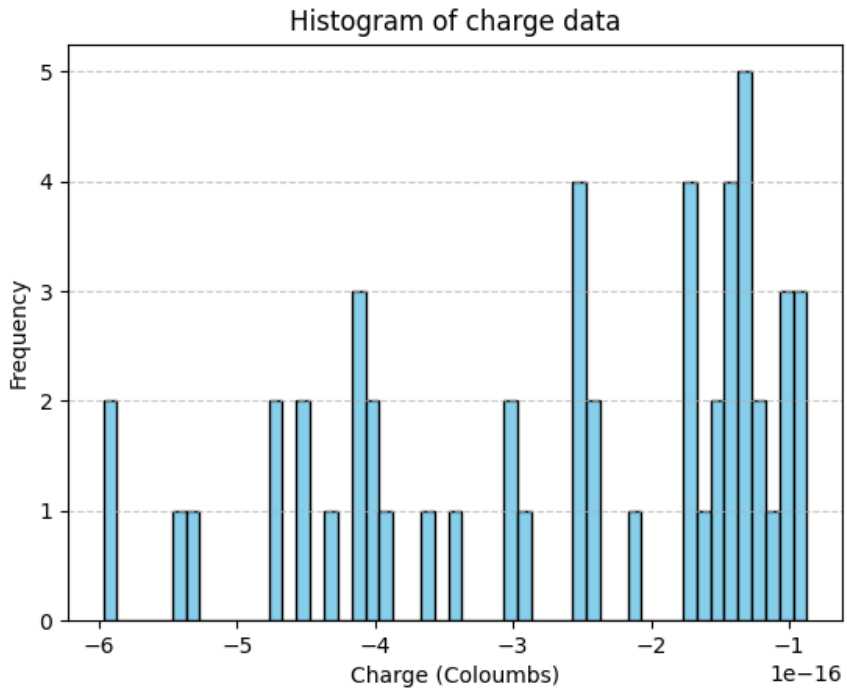
Deriving a formula for the second method follows a similar process - we start first with the formulae for the oil drop rising and falling under applied voltages V_u and V_d:

$$q\frac{V_u}{d} = g(m_{oil} - m_{air}) + 6\pi r\nu v_u$$

$$q\frac{V_d}{d} = 6\pi r\nu v_d - g(m_{oil} - m_{air})$$

(Note here that we can derive the given formula for v_d simply by setting $V_d = 0$.) Substituting for $g(m_{oil} - m_{air})$ in the first equation using the second, we find:

$$q\frac{V_u + V_d}{d} = 6\pi r\nu(v_u + v_d)$$



H

Figure 2: Plot of histogram

Once again, we solve the quadratic equation derived finding the previous method to find the value of r, and find that q is simply:

$$q = \boxed{\frac{4\pi d g(\rho_{oil} - \rho_{air}) r^3}{3V_{stop}}}$$

$$r = \boxed{\sqrt{\frac{9V_{stop}\nu v_d}{2g(V_{stop} - V_d)(\rho_{oil} - \rho_{air})} + (0.815l)^2} - 0.815l}$$

3 Results and discussion

Unless otherwise stated, values of constants are taken from the lab manual [1]. For demonstration purposes, we are showing the results of the trial that result in the largest uncertainty; all other results and respective chi squared values are available in appendix B.

3.1 Histogram

In plotting the histogram, we used bin sizes with widths matching the largest uncertainty in charge (**some number**. The largest uncertainty was $5 \times 10^{-18}C$, so our bin size was $1 \times 10^{-17}C$.

From this histogram in figure 2, distinct peaks can be observed near $q = -1.0 \times 10^{-16}$ C, -2.0×10^{-16} C, -3.0×10^{-16} C, and so on, indicating that the charges occur in discrete, regularly spaced values. This pattern is consistent with the quantization of electric charge, as each droplet carries an integer multiple of some elementary charge e. The spacing between adjacent peaks was estimated to be

$$\Delta q \approx 1.0 \times 10^{-16} \text{ C}.$$

This represents the charge difference between droplets that differ by one unit of net charge. Therefore, the charge per electron is given by

$$e = \frac{\Delta q}{n},$$

where n is the number of excess electrons corresponding to the charge spacing Δq.

Typical oil droplets in this experiment carry on the order of hundreds to thousands of electrons [3]. Assuming $n \approx 600$ electrons per charge interval, the elementary charge is found to be

$$e = \frac{1.0 \times 10^{-16}\ \text{C}}{600} \approx 1.7 \times 10^{-19}\ \text{C}.$$

3.2 Assumptions

For calculation purposes, we ignored the force of buoyancy. The upwards buoyant force is given by

$$F_b = m_{\text{air}} g = \frac{4}{3}\pi r^3 \rho_{\text{air}} g,$$

The gravitational force acting downward is

$$F_g = m_{\text{oil}} g = \frac{4}{3}\pi r^3 \rho_{\text{oil}} g.$$

The net downward force is therefore

$$F_{\text{net}} = F_g - F_b = \frac{4}{3}\pi r^3 (\rho_{\text{oil}} - \rho_{\text{air}}) g.$$

Thus, buoyancy effectively reduces the droplet's weight by the ratio

$$\frac{\rho_{\text{air}}}{\rho_{\text{oil}}}.$$

Plugging in values from the lab manual [1]

$$\frac{\rho_{\text{air}}}{\rho_{\text{oil}}} = \frac{1.204}{875.3} \approx 1.38 \times 10^{-3}.$$

Thus, buoyancy reduces the effective weight by only 0.14%, which is negligible compared to other sources of error and we therefore neglected that force for ease of calculation.

We also assumed the droplets are a perfect sphere. At the microscopic scale, surface tension is the dominant force [4], and it has been empirically shown that spherical droplets are a very good approximation [4]. Additionally, due to the small separation between plates, we assumed a perfectly uniform electric field and ignored all edge effects. Finally, we assumed constant charge for particles, as particles can hold their charge for a significantly longer timescale than the experiment duration (< 1 minute) [5].

3.3 Uncertainty propagation

As per the uncertainty propagation document provided [6], we are ignoring terms that are less than 10% of the largest term. For the velocity, we can take the following uncertainties. From the manufacturer of the apparatus [7], we can take the uncertainty in frame rate to be around $\pm 1\%$, and therefore the uncertainty in time to be $\pm 1\%$. As our data came from the right side device, we are given the uncertainty in calibration to be $520 \pm 1 \frac{\text{px}}{\text{mm}}$. By making our data unidirectional (i.e. using the monotonic filtering algorithm), we would be increasing the slope and therefore slightly, but that is preferable to having noisy bidirectional position data. As well, any data removal was usually for 3 frames at the most. To confirm validity, we compared the raw and filtered data and found that the differences in calculated velocity was $< 3\%$. This is greater than the uncertainty we used of 1%, however by qualitative observation in lab the filtered position values were significantly closer to the actual observed physical position. Additionally, we sometimes observed a variance in position of up to to ± 5 px (i.e. occasionally, observed displacements of up to approximately 5 pixels were

logged as having no displacement due to the tracker box not actually moving), so we will take an uncertainty in position of ±5px. Therefore, the uncertainty in position is 1%, which is a pretty fair value. We calculated the uncertainty in velocity using the standard slope uncertainty equation as follows.

$$\sigma_m = \sqrt{\frac{N}{N-2} \times \frac{\sum_{i=1}^{N} \left(y_i - (mx_i + b)\right)^2}{N \sum_{i=1}^{N} x_i^2 - \left(\sum_{i=1}^{N} x_i\right)^2}}$$

For the other terms, we have the following uncertainties. From the manufacturers specification [3], we can take the uncertainty in the distance between the plates to be ±0.05, or $\frac{0.05}{6} = 0.8\%$. The manufacturer did not specify an uncertainty in voltage, so we will take the uncertainty in the voltage to be one count of the least significant digit, or 0.1 V. At the lowest voltage (200 V), we have $\frac{0.1}{200} = 0.05\%$. As this is less than 10% of the largest uncertainty, we can neglect this [6]. We take our constant values from the lab manual [1]. We take the acceleration of gravity, $g = 9.80 \frac{m}{s^2}$, as exact [8]. The constants used assumed standard atmospheric conditions [9] (1013 hPa, 298 K, etc), which was a pretty reasonable approximation as we conducted our experiments at room temperature. Taking the density of air as 1.204 $\frac{\text{kg}}{\text{m}^3}$, we take the uncertainty to be ±0.08% [10]. We take the density of oil as 875.3 $\frac{kg}{m^3}$ and the uncertainty to be ±0.01% [11]. We take the viscosity of air to be 1.827 $\times 10^{-5}$ Pa s and the uncertainty to be ±0.2%. The actual error would probably be larger, however we did not take pressure, temperature, or humidity values so we could not correct for that quantitatively, and in any case the viscosity should still be a reasonable estimate.
Finding the total uncertainty in charge Q using Method 1, we use the formula provided [6]. First we find the uncertainty of the radius (dr).

$$(dr)^2 = (\frac{dv_d}{4v_d^2}) + (\frac{dV_{stop}^2 + dV_d^2}{4(V_{stop} - V_d)^3}) + (\frac{dV_{stop}^2}{4V_{stop}})$$

$$dr = \sqrt{(\frac{dv_d}{4v_d^2}) + (\frac{dV_{stop}^2 + dV_d^2}{4(V_{stop} - V_d)^3}) + (\frac{dV_{stop}^2}{4V_{stop}})}$$

From this value, the uncertainty in the charge (dQ) can be found as follows (δd is used to represent the uncertainty in the plate width):

$$dQ = Q\sqrt{3(rdr)^2 + (\frac{\delta d}{d})^2 + (\frac{dV_{stop}}{V_{stop}})^2}$$

Putting this all together, for a sample charge (Trial 42), the result is a charge of

$$\boxed{q = (-1.40 \pm 0.01) \times 10^{-16} \text{ C} \qquad (\text{relative uncertainty } = 0.83\%)}$$

The residuals and chi squared for the velocity can be seen in figure 3.

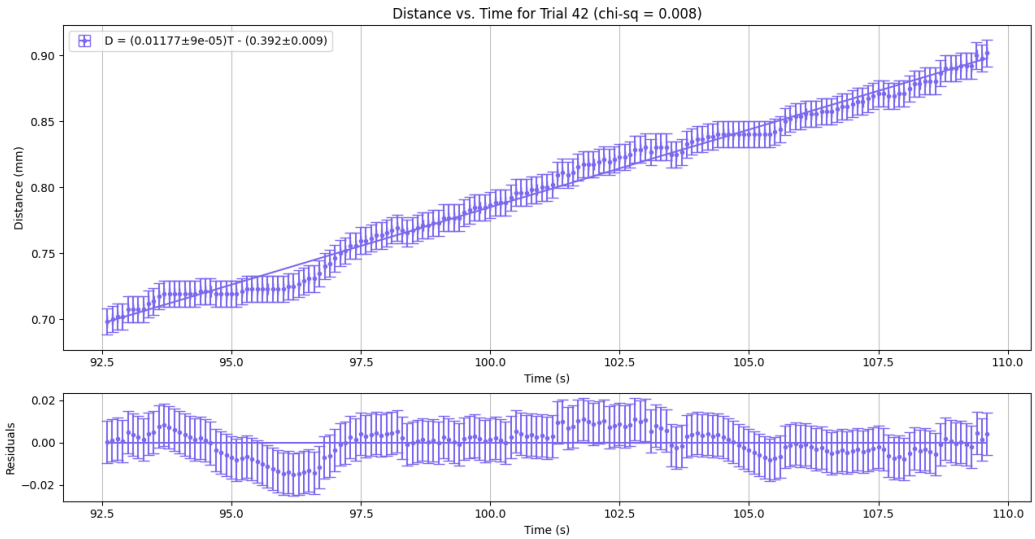


Figure 3: Residuals and chi squared for trial 42

3.4 Comparison to accepted value

The accuracy of our result is found by comparing it to the accepted value. 1.602×10^{-19} C, which is an exact value [12].

$$\text{Percent error} = \frac{|1.7 \times 10^{-19} - 1.602 \times 10^{-19}|}{1.602 \times 10^{-19}} = 6.12\%$$

4 Conclusion

In this set of experiments, we showed that electric charge is quantized. This is shown by comparing the forces on a moving charged oil droplet. The oil droplet experiences 3 forces: a downwards gravitational force, an upwards electrical force, and an viscous friction force which opposed the direction of motion. We used a Leybold-Heraeus apparatus, which adds charge to the oil droplets by friction, and then controls the velocity of the oil droplet via two charged plates powered by a variable power supply. We had 52 trials, and found the greatest common divisor of the charges, which we found to be $1.7 \times 10^{-19}C$. This differed from the accepted value by 6.12%

5 Appendices

5.1 Appendix A: References

References

[1] University of Toronto. *The Millikan Oil-Drop Experiment*. University of Toronto, Toronto, Canada, March 2025. Revisions: 2023 C. Lee; 2018 N. Krasnopolskaia.

[2] mathew. I came up with a single pass $o(n)$ sort algorithm i call stalinsort. `https://mastodon.social/@mathew/100958177234287431`, 10 2018. Accessed: 2025-11-11.

[3] Millikan apparatus. `https://www.leybold-shop.com/physics/physics-equipment/atomic-and-nuclear-physics/fundamentals/millikan-experiment/millikan-apparatus-559412.html`, 2019. Accessed: 2025-11-09; product number 559412, *Leybold / LD Didactic* Physics Equipment.

[4] R. Rajan and A. B. Pandit. Correlations to predict droplet size in ultrasonic atomisation. *Ultrasonics*, 39(4):235–255, 2001.

[5] Joshua Méndez Harper, Dana Harvey, Tianshu Huang, Jack McGrath III, David Meer, and Justin C. Burton. The lifetime of charged dust in the atmosphere. *arXiv preprint arXiv:2206.06848*, 2022. Version v1, 14 Jun 2022.

[6] Uncertainty propagation formulae. Quercus, 2025.

[7] National Instruments. Timed loop - labview structures. `https://www.ni.com/docs/en-US/bundle/labview-api-ref/page/structures/timed-loop.html`, 2024. Accessed: 2025-11-09; documentation for the Timed Loop structure.

[8] National Institute of Standards and Technology (NIST). Codata recommended values of the fundamental physical constants. Web Page, Accessed 2025. URL: `https://physics.nist.gov/cgi-bin/cuu/Value?gn`.

[9] Roland Stull. Standard atmosphere - pressure and density. `https://www.eoas.ubc.ca/courses/atsc113/flying/met_concepts/02-met_concepts/02a-std_atmos-P/index.html`, 2019. Last updated Jan 2019; Accessed: 2025-11-09.

[10] W. F. Koch, R. S. Davis, and V. E. Bower. Direct determination of air density in a balance through artifacts characterized in an evacuated weighing chamber. *J. Res. Natl. Bur. Stand. (U.S.)*, 83(5):407–413, 1978.

[11] Joel T. Park. Application of statistics to oil density measurement. *Measurement*, 30(2):95–103, 2001.

[12] National Institute of Standards and Technology (NIST). Codata recommended value: Elementary charge (e). Web Page, Accessed 2025. URL: `https://physics.nist.gov/cgi-bin/cuu/Value?e`.

5.2 Appendix B: Other velocity and respective chi-Squared Values

Oildrop #	Velocity (m/s)	Uncertainty	Chi squared
1	0.2742	5.00E-05	0.097
2	0.03905	5.00E-05	0.013
3	0.0252	0.0001	0.292
4	0.0342	0.0002	0.001
5	0.03813	4.00E-05	0.022
6	0.0398	0.0001	0.109
7	0.02383	6.00E-05	0.122
8	0.03013	8.00E-05	0.039
9	0.02461	4.00E-05	0.012
10	0.02477	4.00E-05	0.103
11	0.02536	6.00E-05	0.062
12	0.0571	0.0001	0.012
13	0.053	5.00E-05	0.006
14	0.03298	6.00E-05	0.027
15	0.0511	6.00E-05	0.011
16	0.0368	0.0001	0.062
17	0.03278	7.00E-05	0.032
18	0.02499	3.00E-05	0.018
19	0.0814	0.0001	0.003
20	0.03896	8.00E-05	0.039
21	0.0261	0.0001	0.028
22	0.02532	5.00E-05	0.035
23	0.01913	9.00E-05	0.454
24	0.02118	3.00E-05	0.088
25	0.03695	7.00E-05	0.008
26	0.0341	0.0001	0.007
27	0.02333	6.00E-05	0.042
28	0.0262	9.00E-05	0.043
29	0.02522	7.00E-05	0.035
30	0.02715	3.00E-05	0.054
31	0.04154	4.00E-05	0.004
32	0.00885	3.00E-05	0.013
33	0.01191	5.00E-05	0.006

Oildrop #	Velocity (m/s)	Uncertainty	Chi squared
34	0.00818	4.00E-05	0.015
35	0.0097	6.00E-05	0.006
36	0.01069	4.00E-05	0.121
37	0.00701	3.00E-05	0.035
38	0.01646	3.00E-05	0.117
39	0.00903	5.00E-05	0.005
40	0.01959	8.00E-05	0.006
41	0.0105	5.00E-05	0.006
42	0.01177	9.00E-05	0.008
43	0.00669	4.00E-05	0.015
44	0.01228	3.00E-05	0.056
45	0.00914	5.00E-05	0.075
46	0.01408	3.00E-05	0.139
47	0.01057	4.00E-05	0.024
48	0.0084	1.00E-04	0.001
49	0.00626	2.00E-05	0.072
50	0.0109	4.00E-04	0.002
51	0.0364	1.00E-04	0.072
52	0.0446	1.00E-04	0.015

5.3 Appendix C: Charge values

Trial	Charges (C)
1	-4.03E-16
2	-2.97E-16
3	-5.33E-16
4	-3.47E-16
5	-2.57E-16
6	-2.46E-16
7	-4.16E-16
8	-3.98E-16
9	-4.77E-16
10	-4.14E-16
11	-4.14E-16
12	-1.21E-16
13	-1.46E-16
14	-3.00E-16
15	-1.48E-16
16	-2.50E-16
17	-2.54E-16
18	-3.95E-16
19	-1.51E-16
20	-2.10E-16
21	-3.06E-16
22	-4.49E-16
23	-5.97E-16
24	-4.70E-16
25	-2.49E-16
26	-5.91E-16
27	-5.38E-16
28	-4.52E-16
29	-4.29E-16
30	-3.59E-16
31	-2.46E-16
32	-1.72E-16
33	-1.33E-16
34	-1.46E-16
35	-1.33E-16
36	-1.32E-16
37	-1.33E-16
38	-8.76E-17
39	-1.09E-16
40	-1.22E-16
41	-1.30E-16
42	-1.40E-16
43	-1.39E-16
44	-1.76E-16
45	-1.67E-16
46	-1.69E-16
47	-1.72E-16
48	-9.43E-17
49	-1.05E-16
50	-1.05E-16
51	-1.06E-16
52	-9.21E-17

5.4 Appendix D: Code and Data

All code and data available at github.com/jadontsai/phy293_millikan. For completeness, we are pasting our monotonic filtering algorithm algorithm and graphing code.

5.4.1 Equation plotting code

```
import numpy as np
import matplotlib.pyplot as plt
import math

def file_read_1(filename, T, D):
    f = open(filename)
    line_count = 0
    for line in f:
        comma_count = 0
        i = 0
        j = 0
        if(line_count != 0):
            print("b")
            while(i < len(line)):
                print("c")
                if (line[i] == ","):
                    comma_count += 1
                    val = line[j:i]
                    if(comma_count == 1):
                        time = float(val)
                    elif(val != ""):
                        T[comma_count-2].append(time)
                        D[comma_count-2].append(int(val))
                    j = i+1
                i+=1
            val = line[j:i]
            if((val != "\n") and (val != "")):
                T[comma_count-1].append(time)
                D[comma_count-1].append(int(val))
        else:
            print("a")
            while(i < len(line)):
                if (line[i] == ","):
                    T.append([])
                    D.append([])
                i += 1
        line_count+=1

if __name__ == "__main__":
    T = []
    D = []
    file_read_1("help2.csv", T, D)
    for i in range(len(T)):
        aT = np.array(T[i])
        aD = np.array(D[i])
        plt.plot(aT, aD)
        plt.show()
        print(T[i])
```

```
        print(D[i])
```

5.4.2 Filtering algorithm code

```
import pandas as pd
import numpy as np

def stalin_sort_descending_all(csv_file: str, output_file: str):
    df = pd.read_csv(csv_file, skiprows=4) first 4 rows are information

    numeric_cols = df.select_dtypes(include=[np.number]).columns

    if len(numeric_cols) == 0:
        raise ValueError("no number columns")

    sorted_columns = {}

    for col in numeric_cols:
        data = df[col].dropna().tolist()

        if not data:
            continue

        sorted_data = []
        max_value = data[0]
        sorted_data.append(max_value)

        for val in data[1:]:
            if val <= max_value:
                sorted_data.append(val)
                max_value = val

        sorted_columns[f"{col}_stalin_desc"] = sorted_data

    sorted_df = pd.DataFrame(dict([(k, pd.Series(v)) for k, v in sorted_columns.items()]))

    sorted_df.to_csv(output_file, index=False)
    print("stalin sort worked")
stalin_sort_descending_all("raw_data.csv", "sorted.csv")
```

5.4.3 Histogram plotting

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import os

def histogram_with_bin_size(csv_file: str, column_name: str, bin_size: float, output_file: str = None):
    df = pd.read_csv(csv_file)

    if column_name not in df.columns:
        raise ValueError(f"Column '{column_name}' not found")

    data = df[column_name].dropna()
```

```
    if not pd.api.types.is_numeric_dtype(data):
        raise ValueError(f"Column '{column_name}'not numeric")

    data_min, data_max = data.min(), data.max()
    bins = np.arange(data_min, data_max + bin_size, bin_size)

    counts, edges, patches = plt.hist(data, bins=bins, color='skyblue', edgecolor='black')

    max_count_index = np.argmax(counts)
    max_bin_center = (edges[max_count_index] + edges[max_count_index + 1]) / 2
    max_count = counts[max_count_index]

    plt.text(
        max_bin_center,
        max_count + (0.02 * max(counts)),
        f"Peak: {int(max_count)} values\n at {max_bin_center:.2f} +/- {bin_size:.2f}",
        ha='center',
        va='bottom',
        fontsize=9,
        color='black',
        fontweight='bold'
    )

    if not output_file:
        base = os.path.splitext(os.path.basename(csv_file))[0]
        output_file = f"{base}_histogram_{column_name}.png"

    plt.title(f"Histogram of charge data")
    plt.xlabel("Charge (Coloumbs)")
    plt.ylabel("Frequency")
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.savefig(output_file, bbox_inches='tight')
    print("done")
    plt.show()

histogram_with_bin_size("test.csv", "56-83.6", 20, None)
```

5.4.4 Uncertainty code

```
def calculate_unc(x, y, m, b):
    N = len(x)
    s = 0
    s_sq = 0
    for i in range(N):
        s += x[i]
        s_sq += x[i]**2
    delta = N*s_sq - s**2
    var = 0
    for i in range(N):
        var += (y[i] - (m*x[i]+b))**2
    var /= (N-2)
    sm = round_unc((N*var/delta)**(1/2))
    sb = round_unc((var*s_sq/delta)**(1/2))
```

```
    return sm, sb

def round_unc(unc):
    pow = 0
    if(unc > 10):
        i = -1
    elif (unc < 1):
        i = 1
    else:
        return int(unc)
    while((unc > 10) or (unc < 1)):
        unc *= (10)**i
        pow += i
    return int(unc) / 10**pow
```